AI Usage Log - Kaila

CareerNet, database backend. AI tools: Claude and ChatGPT. This log covers the technical questions I worked through with AI and how I checked the answers.

Sprint 1

Sprint 1 - Windows Setup and User Roles

Branches: fix/uvloop-windows and add-role-column

AI Tool: Claude

Task 1 - Python Virtual Environment on Windows

Prompt: "I'm setting up the project on Windows and python3 isn't being recognized, even though Python is installed. Why would that happen, and what's the right way to create and activate a virtual environment here?"

Purpose: Environment setup and troubleshooting

How AI helped:

Explained that the Microsoft Store Python alias can intercept the python3 command on Windows.

Suggested using the py launcher to create the virtual environment.

Showed how to activate it in PowerShell and how to handle the execution policy restriction.

Decision: Accepted

Reflection: I learned that Python commands can differ between operating systems and that virtual environments keep project dependencies separate. I also got more comfortable troubleshooting setup issues instead of immediately reinstalling software.

Task 2 - uvloop Failing to Install

Prompt: "Installing the requirements fails on uvloop for me but works for teammates on Mac. Is uvloop just not supported on Windows? And how can I fix it without breaking it for everyone else?"

Purpose: Dependency management

How AI helped:

Confirmed that uvloop does not support Windows.

Explained that Uvicorn falls back to the default event loop without it.

Suggested a platform marker in requirements.txt so Windows skips uvloop while macOS and Linux still install it.

Decision: Modified to match the project's cross-platform requirements

Reflection: Some installation errors come from the package itself rather than my setup. A platform marker let me fix it for Windows without changing anything for teammates on other systems.

Task 3 - Adding a User Role Column

Prompt: "The app needs to tell job seekers and recruiters apart. What's a good way to add a role to our existing SQLAlchemy User model so it only accepts those two values and lines up with what the frontend already uses?"

Purpose: Database design

How AI helped:

Reviewed the existing User model.

Recommended an enum with the values JOB_SEEKER and RECRUITER.

Pointed out that the values must match the ones the frontend uses.

Suggested Job Seeker as the default so existing users stay valid.

Decision: Modified to match the existing frontend values

Reflection: Enums are useful when a column should only accept specific values. I also saw how a database change has to stay consistent with what other teammates have already built.

Task 4 - Testing the Role Column

Prompt: "How can I actually test the new role field against a database before opening a pull request, without risking the shared one or exposing credentials?"

Purpose: Testing approach

How AI helped:

Suggested a local PostgreSQL database for testing.

Recommended keeping credentials in a .env file that Git ignores.

Explained the difference between reviewing a model change and running it against a real database.

Decision: Accepted, with the testing limitations noted

Reflection: Code review and testing against a database are separate checks, and both matter. Keeping credentials out of the code is part of doing database work safely.

Sprint 2

Sprint 2 - KA-1: Profile, Resume File and Job Tables

Branch: feature/KA-1-profile-resume-job-tables

AI Tool: ChatGPT

Task 5 - Understanding Alembic

Prompt: "I keep seeing Alembic in our setup, but I don't really get what it adds if SQLAlchemy already defines the tables. What's the relationship between the two, and why do migrations matter for a team sharing one database?"

Purpose: Learning migration concepts

How AI helped:

Explained that SQLAlchemy describes the tables while Alembic records and applies changes to them.

Introduced revisions and howAlembic tracks which version a database is at.

Explained why a shared database needs every change tracked.

Decision: Accepted

Reflection: Before this I didn't understand whyAlembic was needed. Now I see the models as the target and migrations as the history of how the database got there, which is what lets every teammate end up with the same schema.

Task 6 - Resume Access: Database or Backend?

Prompt: "Recruiters should see applicants' resumes, but job seekers should only see their own. Is that something the database should enforce, or is it the backend's job? I don't want to over-build the tables."

Purpose: Database design

How AI helped:

Explained that a foreign key links a resume to its owner but doesn't decide who can view it.

Separated storing relationships (the database) from access rules (backend authorization).

Helped me decide whether the tables needed any extra columns for this.

Decision: Kept the original database design

Reflection: Storing data and controlling access to it are separate responsibilities. The permission rules belong in the backend, outside my task, so I kept the tables simple.

Task 7 - Designing the Three Models

Prompt: "I need Profile, ResumeFile and Job models that fit our existing structure. What types and constraints make sense? For example, how should a resume file be stored, and should a job always need a recruiter?"

Purpose: Model implementation

How AI helped:

Walked through primary keys, foreign keys and the unique rule on the profile's user id.

Explained why resume files use LargeBinary.

Explained why recruiter_id has to be nullable, since sample jobs have no recruiter.

Helped choose data types for each column.

Decision: Modified to match the existing project structure

Reflection: I got a clearer picture of how models map to real tables and how constraints keep data consistent. I also got more comfortable building inside the existing structure instead of starting something separate.

Task 8 - Model Import Error

Prompt: "Python says it can't import my ResumeFile model, but the file is right there. What usually causes this?"

Purpose: Debugging

How AI helped:

Spotted that the import didn't match the filename resumeFile.py.

Explained how Python resolves module names, including capitalization.

Suggested a quick import test to confirm the fix.

Decision: Modified to keep my chosen filename

Reflection: A small naming mismatch is enough to break an import. Knowing how Python finds modules makes these errors much faster to track down.

Task 9 - Connecting Alembic to Our Models and Database

Prompt: "How does Alembic actually find our models and connect to Postgres? I want it to use the same database URL as the app instead of a hard-coded one."

Purpose: Configuration

How AI helped:

Explained the roles of alembic.ini and env.py.

Showed how importing the models in App/models/__init__.py lets Base.metadata see them.

Helped point Alembic at the existing environment-based connection.

Decision: Modified to match the existing project configuration

Reflection: If a model isn't imported, Alembic can't see it and the migration comes out empty. I also understood why the database URL should come from environment variables rather than the code.

Task 10 - Tables Exist, but No Migration History

Prompt: "Our database already has all the tables, yet Alembic says there's no migration history and no differences. How can both be true, and how do I start tracking without touching the existing tables?"

Purpose: Database analysis

How AI helped:

Explained that tables existing and Alembic tracking them are two separate things.

Clarified why the schema check found no differences.

Explained the role of an initial migration as a baseline.

Decision: Accepted

Reflection: Having the right tables doesn't mean changes are being tracked. A migration history is what makes the schema reproducible, and a database shared by the whole team needs extra care.

Task 11 - Writing the Initial Migration

Prompt: "Can you help me write an initial migration that matches our existing schema exactly? I want to compare it against the real tables rather than just trusting what gets generated."

Purpose: Migration implementation

How AI helped:

Explained the structure of a revision file, including the upgrade and downgrade functions.

Helped draft the migration for the existing tables.

Guided checking columns, foreign keys, unique constraints and indexes against PostgreSQL.

Helped find and remove a redundant unique constraint on email.

Decision: Modified to match the existing database schema

Reflection: Comparing the generated migration with the real tables caught a duplicate constraint. Generated code needs to be verified, not assumed correct.

Task 12 - Alembic Dialect-Loading Error

Prompt: "Alembic is throwing a dialect-loading error. Is that coming from my models or from the configuration? And can I preview the SQL without running anything on the shared database?"

Purpose: Configuration debugging

How AI helped:

Found a placeholder connection URL in the Alembic configuration.

Fixed the offline migration connection.

Explained the difference between generating SQL and running it, and helped produce an SQL preview.

Decision: Accepted

Reflection: The error came from configuration rather than the models. The SQL preview gave me a way to check a migration without changing the shared database.

Sprint 2 - KA-1 Wrap-up and SA-4: Applications Table

Branch: feature/SA-4-applications-table (Issue #89)

AI Tool: Claude

Task 13 - Applying the KA-1 Migration to the Shared Database

Prompt: "KA-1 is merged, but alembic current prints nothing on Neon. My migration is an initial schema that also creates the users table, which already exists. If I run upgrade head, won't it fail? What's the safest way to get Alembic in sync without touching tables the whole team is using?"

Purpose: Migration strategy on a shared database

How AI helped:

Confirmed that running the migration would fail because it tries to create the users table again.

Suggested checking which tables already existed before deciding anything.

Explained alembic stamp head, which records the migration as applied without running it.

Pointed out that the migration file should stay as it is, so someone setting up a fresh database can still run it normally.

Decision: Accepted, after confirming in pgAdmin that all four tables already existed

Reflection: I learned the difference between applying a migration and recording it. Stamping was the right choice here because the tables were already there; if any had been missing, the fix would have been different. Checking the real state of the database first is what made this a safe decision instead of a guess.

Task 14 - Designing the Applications Table

Prompt: "For SA-4, an application connects a user, a job and the resume they used. How should the foreign keys behave when one of those gets deleted? And what's the cleanest way to stop someone from applying to the same job twice?"

Purpose: Database design

How AI helped:

Suggested a unique constraint on the user and job together, so the database itself blocks duplicate applications.

Explained the delete options: CASCADE removes applications when a user or job is deleted, while SET NULL keeps an application if its resume is deleted.

Pointed out that without SET NULL, deleting a resume used in an application would fail, which would affect the KA-3 delete function.

Decision: Modified. I kept my own version of the model, which uses database-side defaults, rather than the restyled version AI suggested.

Reflection: I hadn't thought about how one table's delete rules affect another feature. Thinking through the delete behaviour made me picture what happens to real data when users or jobs are removed. A constraint in the database is also stronger than a check in the app code, because it can't be skipped.

Task 15 - ImportError on the New Model

Prompt: "Alembic fails with 'cannot import name Application', but the file and class are there. What am I missing?"

Purpose: Debugging

How AI helped:

Asked to see the actual file contents instead of guessing.

Found a typo in the class name (Applicaiton).

Decision: Accepted

Reflection: Looking at what is really in the file is better than assuming what should be there. This was the same kind of naming error as Task 8, so I recognized it faster.

Task 16 - Keeping the Model and Migration in Sync

Prompt: "The migration generated from my model looks right, but I want to make sure it only does what I expect. What should I check before applying it? And after pulling a teammate's change to the user model, how do I know whether it needs its own migration?"

Purpose: Migration review

How AI helped:

Listed what to check in the file: it builds on the KA-1 migration, only creates the applications table, includes the unique rule, and its downgrade only drops that table.

Explained that the model and its migration must match exactly, or the next generated migration picks up unexpected changes.

Suggested running alembic check after pulling, to confirm the teammate's change needed no migration.

Decision: Accepted

Reflection: A generated migration is a draft, not a guarantee. Reading the migration before applying it, and running alembic check after pulling, are habits I'll keep for every change to a shared database.

Sprint 2 - KA-2 to KA-4: Sample Jobs, Resume Storage and Job Search

Branches: feature/KA-2-seed-jobs (Issue #55), feature/KA-3-resume-functions (Issue #56) and feature/KA-4-job-search-query AI Tool: Claude

Task 17 - A Seed Script That's Safe to Rerun

Prompt: "I'm writing a script to fill the jobs table with sample data the team can test search against. Since everyone shares the same database, how do I make sure running it twice doesn't double everything? And what makes sample data actually useful for testing?"

Purpose: Script design and test data

How AI helped:

Suggested checking for an existing job with the same title and company before adding it, so reruns skip duplicates.

Laid out the data so every category and location has the same number of jobs, several titles contain "Developer", and one job is closed.

Used our real Job columns and the project's existing database session, after I shared them.

Rolled back on errors, so a failed run doesn't leave half the data behind.

Decision: Accepted, after checking the data against the guide's requirements

Reflection: Making a script safe to rerun matters much more on a shared database. The sample data was also designed around the cases KA-4's search needs to handle, like keyword matches, filters and a closed job, so it's built for testing rather than just filling rows.

Task 18 - Resume Functions That Keep Users' Files Private

Prompt: "I need save, list, get, replace and delete functions for resumes that Lamees can call from her endpoints. How do I make sure a user can never reach someone else's resume? And since the files are stored in the table, how do I keep listing resumes from loading megabytes of data?"

Purpose: Data access design

How AI helped:

Had every function take the owner's user id and filter by it, so another user's resume returns nothing, as if it didn't exist.

Used SQLAlchemy's defer option so listing resumes skips the file data column.

Committed after every save, replace and delete, to avoid changes that seem saved but disappear.

Put the 5 MB limit and the allowed file types in a shared rules file for the backend and frontend.

Decision: Accepted

Reflection: Building ownership into every query means the endpoints don't have to remember to check it. I also learned that a large column like file data can be left out of a query, which matters once real resumes are stored.

Task 19 - Testing Data Isolation Between Users

Prompt: "How can I properly test these functions before handing them off? I especially want to prove one user can't touch another's resume, without leaving test data in the shared database."

Purpose: Testing

How AI helped:

Wrote a throwaway script that uses two real users and runs every function as both of them.

Checked that every attempt by one user to read, replace or delete the other's resume was refused.

Deleted the test resume at the end, so nothing was left behind.

Decision: Accepted. All 12 checks passed, and I kept the test script out of the repository.

Reflection: Testing the failure cases, not just the normal path, is what actually proves the privacy rule works. Cleaning up afterwards is part of testing responsibly on a shared database.

Task 20 - Building a Flexible Job Search

Prompt: "For KA-4, the search should take a keyword, location and category, but any of them can be missing. How do I build the query so missing filters are just skipped? And how do I return one page of results while still knowing how many matches there are in total?"

Purpose: Query design

How AI helped:

Built the query step by step, adding each filter only when a value is given.

Used a case-insensitive search across the title, company and description for the keyword, and case-insensitive exact matches for location and category.

Counted the total before limiting the results to one page, so the total isn't capped at the page size.

Hid closed jobs by default, with an option to include them later for recruiter pages.

Decision: Accepted. I asked for simple comments instead of docstrings, to match the style of our other files.

Reflection: Building the query in steps made the "skip missing filters" rule simple. The counting order was a subtle bug I wouldn't have noticed on my own, since a capped total still looks believable.

Task 21 - Testing Search Against Sample Data

Prompt: "How can I check the search actually behaves the way the guide describes, using the sample jobs, without changing anything in the database?"

Purpose: Testing

How AI helped:

Wrote a read-only throwaway script covering the guide's four cases: no filters, a keyword, location plus category, and no matches.

Added checks for paging, newest-first order, hidden closed jobs and job lookup.

Printed what each search returned, so I could check the results myself.

Decision: Accepted. All 16 checks passed, and I kept the test script out of the repository.

Reflection: Seeing the actual results helped me understand them. Searching "developer" also matched two designer jobs, because their descriptions mention working with developers. That's correct, since the keyword searches descriptions too. The KA-2 sample data made this much easier to test.

Sprint 3

Sprint 3 - Setup and KA-9: One Login Helper

Branch: feature/KA-9-single-login-helper

AI Tool: Claude

Task 22 - Setting Up the Project on a Second Computer

Prompt: "I'm on a different computer and want to install requirements.txt before starting KA-9, but the venv won't activate. What's the right order to set this up, and what should I check before committing from this machine?"

Purpose: Environment setup

How AI helped:

Explained that my first install went into the computer's main Python because the venv didn't exist yet, and walked me through creating .venv.

Explained the PowerShell execution policy and how to allow scripts for my user only.

Pointed out that the folder path had a different username than last time, so I should check which name and email my commits would use.

Noticed that my Git email was already my GitHub no-reply address, which links commits to my account, after I had replaced it with my Gmail.

Decision: Modified. I switched my Git email back to the GitHub no-reply address.

Reflection: Individual contribution is read from the commit history, so a wrong Git identity could have made my work look like someone else's. Setting up a new machine is a good moment to check that, not something to skip.

Task 23 - Looking Before Changing: Which Path for KA-9?

Prompt: "Before I change either login helper, can you help me read the step 0 results and figure out which path actually fits our code?"

Purpose: Code investigation

How AI helped:

Gave me PowerShell versions of the guide's step 0 commands (git grep instead of grep).

Read my results with me: four routers used deps.py, two used dependencies.py, and the only frontend match for "Not logged in" was a comment.

Explained that no test checks the message text, only the status code, so no existing test would need to change.

Recommended path C then A, because the missing-header case was the only real difference between the two helpers.

Decision: Accepted, after checking the grep output myself.

Reflection: The investigation took less time than the change, and it decided everything. Knowing that nothing depended on the message text made the change much less risky than it first looked.

Task 24 - Merging the Helpers and Proving Nothing Broke

Prompt: "Can you help me make dependencies.py the only helper and write tests that prove every bad login gets the same answer? And did my change cause the app to crash?"

Purpose: Implementation, testing and debugging

How AI helped:

Rewrote get_current_user in dependencies.py with HTTPBearer(auto_error=False), so a missing header gets our own "Not logged in." instead of the library's message.

Wrote 17 tests that try three bad headers, an expired token and a deleted account on three different routes, plus the 403 for a job seeker on a recruiter route.

Explained that the crash came from a missing .env on this computer, not from my code, and gave me a way to check the app starts using a throwaway local database.

Corrected its own test count when I got 124 instead of the 125 it predicted.

Decision: Accepted. 124 tests passed and the app started with no import errors.

Reflection: Testing every combination is what proves the messages are really the same everywhere. I also learned to read an error carefully before assuming my change caused it: the traceback pointed at DATABASE_URL, not at the login helper.

Sprint 3 - KA-6: Notifications Table and Migration

Branch: feature/KA-6-notifications-table

AI Tool: Claude

Task 25 - Designing the Notifications Table

Prompt: "How should the notifications table behave when a user or application is deleted, and how does the event key stop duplicate reminders? I can't connect to the database from this computer yet, so how do I write the migration?"

Purpose: Database design and migration

How AI helped:

Explained the delete rules: CASCADE removes a user's notifications with the user, while SET NULL keeps a notification but clears its link if the application is deleted.

Explained that a unique rule on user and event key lets the database itself refuse a duplicate reminder, while notifications with no event key are still allowed.

Wrote the migration by hand, since autogenerate needs a database connection, and suggested alembic check afterwards to confirm it matches the models.

Pointed out that SQLite only follows delete rules when foreign keys are switched on, so the delete test turns them on and off again.

Decision: Accepted. I asked whether the docstring at the top of the migration was necessary, and kept it to match the other migrations.

Reflection: This was the same kind of thinking as the applications table in SA-4: what happens to real data when something is deleted. Writing the migration by hand also made me read every line of it instead of trusting a generated file.

Task 26 - Connecting to Neon from a New Computer

Prompt: "My .env is empty and I don't remember where the secret key is. Is it the same for everyone, and why is Alembic using SQLite instead of Neon?"

Purpose: Configuration and debugging

How AI helped:

Found in our README that each teammate makes their own JWT secret, and that the values I remembered from config.py and conftest.py are only fallbacks and test values.

Gave me one command to write the .env and generate the secret, and to check that Git ignores the file.

Explained that the temporary SQLite setting from the earlier startup check was still active in my terminal, because Ctrl+C stopped the cleanup lines from running.

Reminded me that the database link contains the password and shouldn't be pasted into chats or commits.

Decision: Accepted

Reflection: Settings in the terminal take priority over the .env file, which is why Alembic was talking to the wrong database. Checking alembic current before migrating caught it, which is exactly why that check comes first.

Task 27 - Running the Migration on the Shared Database

Prompt: "The preview shows every migration from the start. Will it rebuild our tables? And if it's already in the database, why do I still need a pull request?"

Purpose: Migration review on a shared database

How AI helped:

Explained that --sql mode doesn't look at the database, so it prints the whole chain, while the real upgrade only runs what is missing.

Pointed me to the part that mattered: only the notifications table, its index and interview_at, with nothing dropped.

Explained that the database and the code are separate: the table now exists on Neon, but the model and migration files only existed on my computer, so teammates couldn't use them and Alembic would error for them.

Decision: Accepted. I told the team before running it, and afterwards alembic current showed 1dae1f3df735 and alembic check found no differences.

Reflection: I'm glad I asked before running it. A migration has two halves, the change in the database and the file in the repository, and the team needs both.

Sprint 3 - KA-7: Notification Service

Branch: feature/KA-7-notification-service

AI Tool: Claude

Task 28 - Notification Functions Other People Will Build On

Prompt: "Lamees and Youssef will call these functions, so why shouldn't they commit on their own? And how can reminders work without a background timer?"

Purpose: Service design and testing

How AI helped:

Explained the no-commit rule: the caller commits once, so a notification is only saved if the change that caused it is saved too.

Explained that reminders are created when the user opens their notifications, and that putting the interview time in the event key means a rescheduled interview gets a new reminder.

Pointed out that SQLite returns dates without a time zone, so those are treated as UTC before comparing.

Wrote 15 tests, including that a rollback removes an unsaved notification and that a stranger can't mark someone else's notification as read.

Decision: Accepted. 127 tests passed, and I posted the function names for Lamees and Youssef before the merge.

Reflection: Writing code that others depend on means thinking about how they'll use it, not just whether it works. Opening the pull request early lets them start from the function names instead of waiting for me.

Task 29 - Writing Acceptance Tests Before the Feature Exists

Prompt: "The notification endpoints and screens aren't built yet. How do I write US-5 acceptance tests that are still useful now and can be run later?"

Purpose: Acceptance testing

How AI helped:

Explained the Given / When / Then format from our Team Guide and that each scenario is its own issue.

Drafted the three US-5 scenarios: a status change notifies the job seeker, a new application notifies the recruiter, and an interview within 24 hours creates a reminder.

Suggested pointing each draft at my existing service tests for now, and marking the app-level run as waiting for LA-7, LA-8, YO-7 and DA-7.

Decision: Kept the drafts, but decided to post them once Sayf creates the US-5 story issue, so they can link to it.

Reflection: Writing the scenarios early made me think about the feature from the user's side, not just the database side. It also showed which parts depend on my teammates' tasks.

Sprint 3 - KA-8: Demo Data Script

Branch: feature/KA-8-demo-data

AI Tool: Claude

Task 30 - A Demo Script That Can't Hurt the Shared Database

Prompt: "This script writes to Neon, which the whole team uses. How do I make sure it can't run by accident or add everything twice, and that the demo actually shows every status and a reminder?"

Purpose: Script design and test data

How AI helped:

Followed the style of my Sprint 2 seed script and reused my KA-7 functions to create the notifications.

Added a check that stops if the demo recruiter already exists, and a prompt that shows the database host and only continues if I type YES.

Spread 12 applications over the last 30 days with all five statuses, and put one interview less than 24 hours away so a reminder appears.

Used @example.com emails, because our login validation rejects reserved domains like .test.

Replaced the plain job list with more interesting companies when I asked for cooler jobs.

Decision: Modified. I asked for a more interesting set of jobs, then accepted it.

Reflection: The YES prompt did its job right away: I typed "yes" in lowercase and it cancelled, which is exactly the kind of mistake it's there to catch. Running it twice and seeing "Nothing was added" proved it's safe for the team to use.

Task 31 - Why Some Pull Requests Weren't Being Checked

Prompt: "KA-9 and KA-6 show the CI checks, but KA-7 and KA-8 don't. Why, and how do I fix it without breaking anything?"

Purpose: CI and GitHub workflow

How AI helped:

Read our CI workflow and found it only runs on pull requests into main, while KA-7 and KA-8 were based on my KA-6 and KA-7 branches.

Gave two options: retarget the pull requests to main now, or wait until the earlier ones are merged.

Advised not to change the CI file, since it belongs to Sayf's task.

Decision: Accepted

Reflection: Stacking pull requests on each other kept the diffs small, but it skipped the CI checks. Knowing when CI runs helps me make sure every change is actually tested before it's merged.

Sprint 3 - KA-10: Requirements and README Cleanup

Branch: feature/KA-10-requirements-readme

AI Tool: Claude

Task 32 - Removing a Package That Turned Out to Be Needed

Prompt: "The guide says to remove httpx2 if nothing imports it, and grep finds nothing. But the clean-environment test shows a warning telling me to install httpx2. Which one should we keep?"

Purpose: Dependency cleanup

How AI helped:

Found that pytest was listed twice and that grep shows no direct use of httpx2 in our code.

Gave me commands to prove the file is complete in a brand-new environment on Windows.

Read the warning with me: the test client from FastAPI and Starlette now expects httpx2, so httpx was the deprecated one.

Had me test the reverse change in a second clean environment, which passed with no warning.

Rewrote the README Features list so it only claims what exists, and added the API and Running Tests sections.

Decision: Modified. I kept httpx2 and removed httpx, the opposite of the guide's suggestion, because the clean test showed it.

Reflection: Grep only finds what our own code imports, not what a library uses behind the scenes. Testing in a clean environment caught something a quick search missed, and reading the warning instead of ignoring it gave a cleaner result.

Final Outcome

Sprint 1: Fixed the uvloop installation issue on Windows with a platform marker, without affecting macOS or Linux.

Sprint 1: Added the user role column with the JOB_SEEKER and RECRUITER values, defaulting to Job Seeker.

Sprint 2 (KA-1): Built the Profile, ResumeFile and Job models and the initial migration. Merged, and the migration is stamped on Neon.

Sprint 2 (SA-4): Built the applications table and its migration, applied to Neon. Pull request open.

Sprint 2 (KA-2): Built the sample-jobs seed script. 18 jobs added, and rerunning adds none. Pull request open.

Sprint 2 (KA-3): Built the resume storage functions. 12 of 12 checks passed. Pull request open.

Sprint 2 (KA-4): Built the job search and lookup functions. 16 of 16 checks passed. Pull request open.

Sprint 3 (KA-9): Merged the two login helpers into one. Every protected route now answers 401 "Not logged in." 17 new tests, 124 passing. Pull request open.

Sprint 3 (KA-6): Built the notifications table and the interview_at column, with migration 1dae1f3df735 applied to Neon. 5 new tests. Pull request open.

Sprint 3 (KA-7): Built the notification service for Lamees and Youssef. 15 new tests, 127 passing. Pull request #124 open.

Sprint 3 (KA-8): Built the demo data script. Added 2 accounts, 12 jobs, 12 applications and 14 notifications on Neon; rerunning adds nothing. Pull request #125 open.

Sprint 3 (KA-10): Cleaned requirements.txt (one line per package, httpx2 kept after a clean-environment test) and made the README match the real features. 107 tests pass with no warnings. Pull request open.

Overall Reflection: Over these three sprints, AI was most useful for understanding unfamiliar tools like Alembic, and for thinking through design decisions before writing code, such as how tables should behave when data is deleted and how to keep users' data private. The biggest thing I learned is to treat AI suggestions as a starting point and check them against reality. I inspected the real tables in pgAdmin before stamping a migration, read every generated migration before applying it, ran alembic check after pulling teammates' changes, and tested failure cases as well as the normal path. Working on a database the whole team shares made me much more careful about checking before changing anything. In Sprint 3 I also learned to read errors before assuming my change caused them, and that a migration has two halves: the change in the database and the file in the repository, and the team needs both. Building the notification service for teammates made me think about how other people will use my code, not just whether it works. Testing in a clean environment also caught a dependency that a quick search missed.